JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Annotation Interface Deprecated

```
@Documented
@Retention(RUNTIME)
@Target({CONSTRUCTOR,FIELD,LOCAL_VARIABLE,METHOD,PACKAGE,MODULE,PARAMETER,TYPE})
public @interface Deprecated
```

A program element annotated `@Deprecated` is one that programmers are discouraged from using. An element may be deprecated for any of several reasons, for example, its usage is likely to lead to errors; it may be changed incompatibly or removed in a future version; it has been superseded by a newer, usually preferable alternative; or it is obsolete.

Compilers issue warnings when a deprecated program element is used or overridden in non-deprecated code. Use of the `@Deprecated` annotation on a local variable declaration or on a parameter declaration or a package declaration has no effect on the warnings issued by a compiler.

When a module is deprecated, the use of that module in `requires`, but not in `exports` or `opens` clauses causes a warning to be issued. A module being deprecated does *not* cause warnings to be issued for uses of types within the module.

This annotation type has a string-valued element `since`. The value of this element indicates the version in which the annotated program element was first deprecated.

This annotation type has a boolean-valued element `forRemoval`. A value of `true` indicates intent to remove the annotated program element in a future version. A value of `false` indicates that use of the annotated program element is discouraged, but at the time the program element was annotated, there was no specific intent to remove it.

API Note:

> It is strongly recommended that the reason for deprecating a program element be explained in the documentation, using the `@deprecated` javadoc tag. The documentation should also suggest and link to a recommended replacement API, if applicable. A replacement API often has subtly different semantics, so such issues should be discussed as well.
>
> It is recommended that a `since` value be provided with all newly annotated program elements. Note that `since` cannot be mandatory, as there are many existing annotations that lack this element value.
>
> There is no defined order among annotation elements. As a matter of style, the `since` element should be placed first.

The `@Deprecated` annotation should always be present if the `@deprecated` javadoc tag is present, and vice-versa.

See *Java Language Specification*:

9.6.4.6 @Deprecated

Since:

1.5

Optional Element Summary

Optional Elements

Modifier and Type	Optional Element	Description
`boolean`	**`forRemoval`**	Indicates whether the annotated element is subject to removal in a future version.
`String`	**`since`**	Returns the version in which the annotated element became deprecated.

Element Details

since

`String since`

Returns the version in which the annotated element became deprecated. The version string is in the same format and namespace as the value of the `@since` javadoc tag. The default value is the empty string.

Returns:

the version string

Since:

9

Default:

""

forRemoval

`boolean forRemoval`

Indicates whether the annotated element is subject to removal in a future version. The default value is `false`.

Returns:

whether the element is subject to removal

Since:

9

Default:

`false`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.